GUÍA DE COMANDOS IMPRESCINDIBLES DE CLAUDE

Trabaja con la herramienta de una forma más ágil, ordenada y autónoma

BIG school

EL LADO DEL MAL

ANTES DE EMPEZAR

Claude Code puede hacer mucho más que responder preguntas dentro de una terminal.

Puede leer los archivos de un proyecto, entender su estructura, ejecutar herramientas, modificar código, mantener el contexto de una sesión y ayudarte a documentar el trabajo realizado.

Esa capacidad resulta especialmente útil durante una práctica de hacking ético.

Una auditoría puede incluir reconocimiento, enumeración, análisis de código, revisión de peticiones, creación de scripts, documentación de hallazgos y muchas decisiones intermedias. Si la sesión crece sin ningún orden, es fácil perder el contexto, repetir instrucciones o dejar que Claude avance sin revisar suficientemente lo que está haciendo.

Aquí es donde entran los comandos de Claude Code.
Los comandos son atajos que te permiten controlar la sesión sin tener que explicar cada vez lo que necesitas. Puedes utilizarlos para:

- Preparar el contexto de un laboratorio.
- Decidir qué herramientas puede ejecutar Claude.
- Pedirle que planifique antes de actuar.

- Revisar cuánto contexto está consumiendo la sesión.
- Comprobar los cambios que ha realizado.
- Volver a un punto anterior si algo no sale como esperabas.
- Guardar el trabajo y retomarlo más adelante.

Esta guía sigue el recorrido habitual de una práctica:

PREPARAR → INVESTIGAR → REVISAR → GUARDAR → RECUPERAR

No necesitas memorizar todos los comandos.

La idea es que entiendas qué problema resuelve cada uno y tengas esta guía cerca cuando necesites consultarlo.

Al terminar, no solo sabrás qué hace /init, /compact o /rewind.

También tendrás una forma más ordenada de trabajar con Claude Code sin perder el control de la auditoría.

1. CÓMO UTILIZAR ESTA GUÍA

1. Los comandos empiezan por /
Los comandos se escriben al principio del mensaje dentro de una sesión activa de Claude Code.

Por ejemplo:
/plan

Al escribir /, Claude Code muestra los comandos disponibles en tu instalación. La lista puede incluir comandos integrados, Skills, plugins o instrucciones expuestas por servidores MCP, así que puede variar según la configuración.

2. Un comando y una instrucción no hacen lo mismo
El comando controla cómo funciona la sesión.

La instrucción explica qué tarea quieres realizar.

Por ejemplo:
/plan

Analiza este laboratorio y propón un proceso de reconocimiento. No ejecutes ninguna herramienta todavía.

/plan activa una forma de trabajo.

La frase posterior define el objetivo concreto.

3. Cómo leer la sintaxis
Cuando veas:
/model [modelo] lo que aparece entre corchetes es un dato que puedes añadir.

Ejemplo: /model opus

Cuando veas: /add-dir <ruta>
debes sustituir <ruta> por la ubicación real.

Ejemplo: /add-dir /home/kali/wordlists

Cuando aparezca una opción como: [--fix]
significa que es opcional. No necesitas utilizarla siempre.

4. Comprueba qué tienes disponible
Las funciones de Claude Code pueden cambiar con las actualizaciones y con la configuración de cada usuario.

Antes de asumir que un comando está disponible:

1. Escribe /.
2. Empieza a escribir su nombre.
3. Comprueba que aparece en la lista.
4. Revisa qué argumentos admite.

5. Trabaja siempre dentro de un entorno autorizado
Utiliza Claude Code únicamente sobre:

- Sistemas propios.
- Laboratorios vulnerables.
- Entornos en los que tengas permiso explícito.
- Programas de divulgación que definan claramente su alcance.

Los comandos te ayudan a controlar Claude.
No amplían el alcance legal de una auditoría.

2. PRIMEROS PASOS: PREPARA TU ENTORNO DE TRABAJO

COMANDO	PARA QUÉ SIRVE
`/init`	Genera un **CLAUDE.md** inicial con contexto del proyecto (estructura del laboratorio, convenciones, notas). Claude lo lee en cada sesión posterior.
`/permissions`	Abre el panel donde apruebas, preguntas o deniegas el uso de cada herramienta (ej. ejecutar **sqlmap**, **nmap** o **curl** por Bash). Alias: `/allowed-tools`
`/mcp`	Gestiona las conexiones a servidores MCP (por ejemplo, integraciones con Burp Suite) y su autenticación.
`/agents`	Recuerda cómo pedirle a Claude que cree o gestione subagentes especializados (reconocimiento, explotación, informe...).

En la práctica: al desplegar un laboratorio nuevo de DockerLabs, ejecuta /init para que Claude entienda el contexto del proyecto, y revisa /permissions para decidir qué comandos de red o de shell puede lanzar sin pedirte confirmación cada vez.

3. DURANTE LA AUDITORÍA: INVESTIGA Y MANTÉN EL CONTROL

Una auditoría se alarga: reconocimiento, enumeración, explotación, escalada. Estos comandos te ayudan a avanzar sin perder el hilo ni saturar el contexto.

COMANDO	PARA QUÉ SIRVE
`/plan`	Entra en modo Plan antes de un cambio grande: Claude propone los pasos antes de ejecutar nada.
`/model [modelo]`	Cambia el modelo de IA. Sin argumento abre un selector.
`/effort [nivel]`	Ajusta cuánto "razona" el modelo (low, medium, high, xhigh, max).
`/context [all]`	Muestra, como una cuadrícula de colores, qué está ocupando la ventana de contexto.
`/compact [instrucciones]`	Resume la conversación para liberar contexto sin perder el hilo.
`/btw <pregunta>`	Hace una pregunta rápida sin añadirla a la conversación principal.
`/add-dir <ruta>`	Añade un directorio adicional para que Claude pueda leer archivos fuera del proyecto actual.
`/cd <ruta>`	Mueve la sesión a otro directorio de trabajo, conservando la caché de prompts.

En la práctica: si la sesión se alarga mucho (muchas peticiones de Burp, salidas de sqlmap...), revisa /context y lanza /compact antes de que Claude empiece a "olvidar" el principio de la auditoría.

4. REVISAR HALLAZGOS: DIFF, CODE-REVIEW Y SECURITY-REVIEW

Antes de dar por cerrada una fase (o de escribir el informe), estos comandos revisan lo que ha cambiado o lo que has construido.

COMANDO	PARA QUÉ SIRVE
`/diff`	Abre un visor interactivo con los cambios sin confirmar, turno a turno.
`/code-review [nivel] [--fix]`	Revisa el diff en busca de errores de corrección y limpiezas. Con **--fix** aplica los hallazgos a tu árbol de trabajo.
`/review`	Ejecuta una revisión de solo lectura sobre una pull request de GitHub.
`/security-review`	Revisión específica del diff en busca de vulnerabilidades de seguridad.

En la práctica: si le pides a Claude que te ayude a escribir un script de explotación o un exploit en Python, cierra esa tarea con /security-review antes de ejecutarlo: te ayuda a detectar fallos introducidos sin querer, además de los que estás buscando a propósito en el objetivo.

5. ENTRE SESIONES: GUARDA, RETOMA Y RAMIFICA

Una auditoría rara vez se resuelve de una sentada. Estos comandos gestionan cómo entras y sales de cada sesión.

COMANDO	PARA QUÉ SIRVE
`/clear [nombre]`	Empieza una conversación nueva con contexto vacío, conservando la memoria del proyecto. Alias: `/reset` `/new`
`/resume`	Vuelve a una conversación anterior desde el selector.
`/branch [nombre]`	Crea una rama de la conversación actual para probar un camino distinto sin perder la original.
`/export [archivo]`	Exporta la conversación completa como texto plano.
`/cost`	Alias de `/usage` : muestra el consumo de tokens y el coste estimado de la sesión.

En la práctica: cuando termines la fase de reconocimiento de una máquina, usa /clear informe-reconocimiento para dejarla etiquetada y empezar limpio la fase de explotación, sin perder la posibilidad de volver atrás con /resume.

6. CUANDO ALGO FALLA: DIAGNÓSTICO Y RECUPERACIÓN

Si la sesión se comporta raro, o Claude ha tocado algo que no debía, estos comandos te sacan del apuro.

COMANDO	PARA QUÉ SIRVE
/doctor	Verificación completa de tu instalación: configuración, skills sin usar, MCP mal conectados. Alias: /checkup
/debug [descripción]	Activa el registro de depuración y ayuda a investigar un problema concreto.
/rewind	Revierte el código y/o la conversación a un checkpoint anterior.
/feedback [informe]	Reporta un error con el contexto de la sesión adjunto. Alias: /bug /share

En la práctica: si Claude ejecuta un payload que deja el laboratorio en un estado raro, /rewind es tu botón de pánico antes de volver a desplegar la máquina desde cero.

COMANDO	PARA QUÉ SIRVE
`/doctor`	Verificación completa de tu instalación: configuración, skills sin usar, MCP mal conectados. Alias: `/checkup`
`/debug [descripción]`	Activa el registro de depuración y ayuda a investigar un problema concreto.
`/rewind`	Revierte el código y/o la conversación a un checkpoint anterior.
`/feedback [informe]`	Reporta un error con el contexto de la sesión adjunto. Alias: `/bug` `/share`

En la práctica: si Claude ejecuta un payload que deja el laboratorio en un estado raro, /rewind es tu botón de pánico antes de volver a desplegar la máquina desde cero.

7. CHULETA RÁPIDA

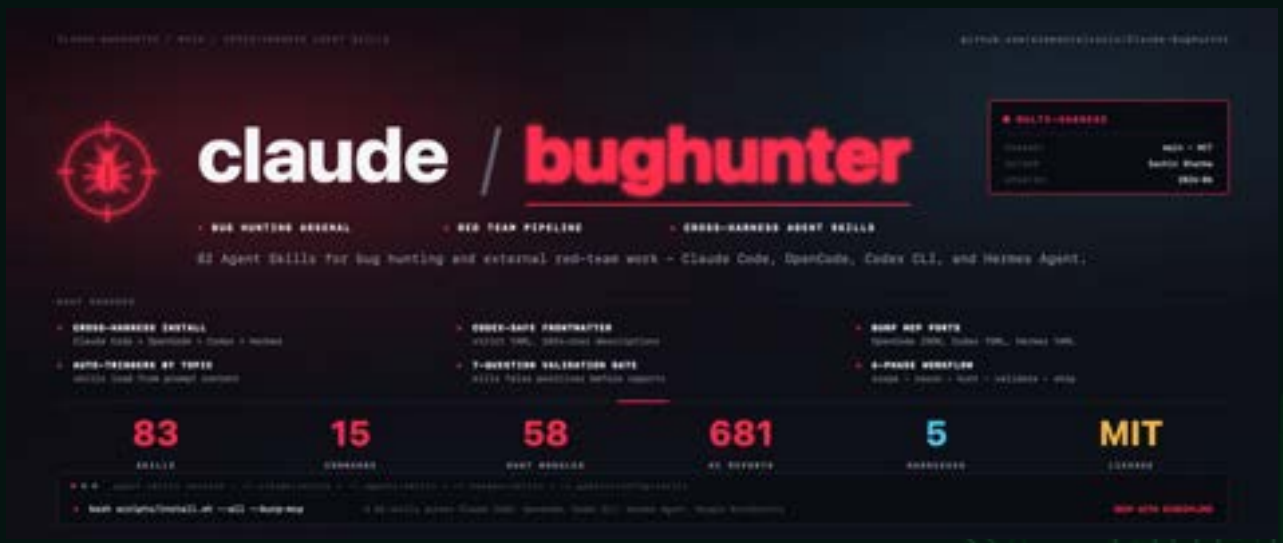
Para empezar, quédate con esta lógica:

FASE	COMANDOS CLAVE
Preparar el entorno	`/init` `/permissions` `/mcp` `/agents`
Investigar y avanzar	`/plan` `/model` `/effort` `/context` `/compact` `/btw` `/add-dir` `/cd`
Revisar hallazgos	`/diff` `/code-review` `/review` `/security-review`
Entre sesiones	`/clear` `/resume` `/branch` `/export` `/cost`
Si algo falla	`/doctor` `/debug` `/rewind` `/feedback`

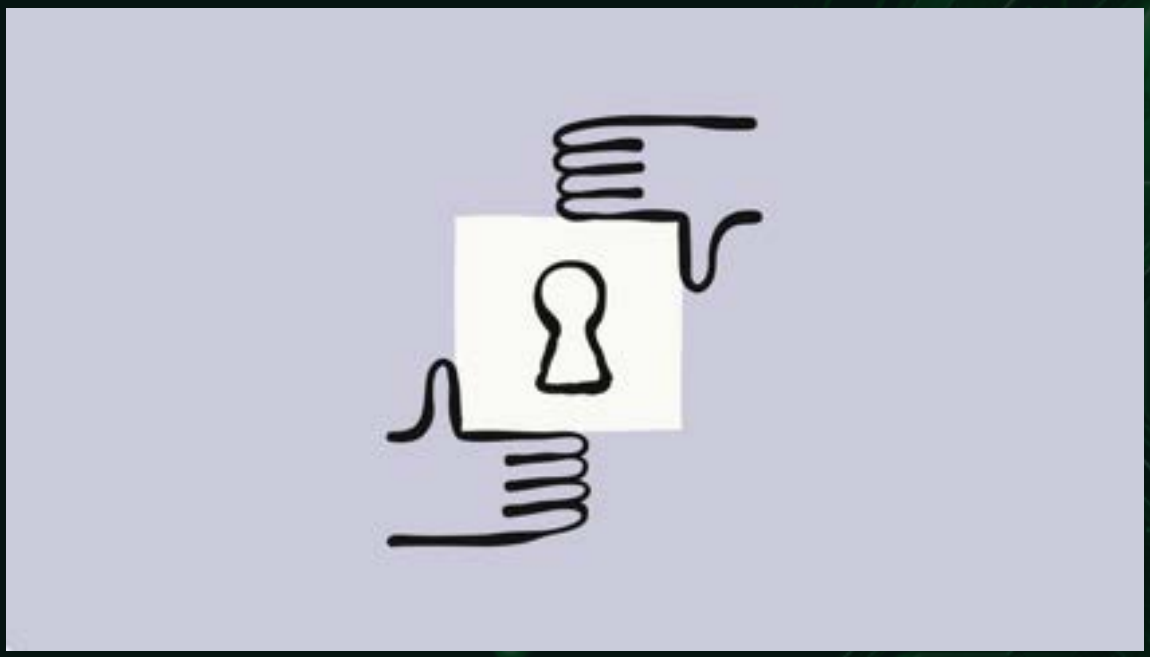
8. SKILLS CLAUDE CODE PARA HACKEAR

Para empezar, quédate con esta lógica:

REPOSITORIO DE GIT HUB



PROGRAMA EN CIBERSEGURIDAD DE ANTHROPIC



Un último atajo: escribe / seguido de unas letras en cualquier momento para filtrar la lista completa y comprobar si existe un comando para lo que necesitas, no hace falta memorizarlos todos.

CONCLUSIÓN

No hace falta dominar todos estos comandos de golpe. La idea es la misma que con el resto del curso: ve soltándolos poco a poco, uno por laboratorio, hasta que formen parte natural de tu flujo de auditoría.

Si ya has configurado tu entorno con la guía de OpenCode y tienes tu laboratorio vulnerable desplegado, este es el momento de meter estos comandos en la ecuación: deja que Claude te ayude en el reconocimiento y la documentación mientras tú te centras en el criterio técnico — justo lo que vamos a ver en la próxima sesión en directo del curso.

BIG school

CONVIÉRTETE EN UN EXPERTO POTENCIADO CON IA

INSCRÍBETE EN EL MÁSTER EN CIBERSEGURIDAD Y HACKING ÉTICO

MATRICÚLATE AQUÍ